Windowed-Sinc Filters

Digital Signal Processing

August 25, 2026

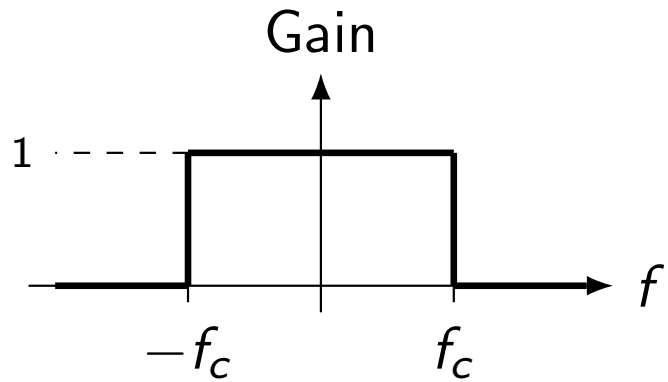
Why Windowed-Sinc Filters?

- Used to separate one band of frequencies from another (low-pass, high-pass, band-pass, band-reject)
- Very stable and predictable — they produce few surprises
- Can be pushed to **extremely high** frequency-domain performance

- That performance comes at a cost:
  - Poor **time-domain** behavior: excessive ripple and overshoot in the step response
- Implemented by standard convolution: easy to program, but slow
  - The FFT can dramatically speed up execution (covered later)

The Ideal Low-Pass Frequency Response

- All frequencies **below** the cutoff f_c pass with unity amplitude
- All frequencies **above** f_c are completely blocked
- The passband is perfectly flat
- Stopband attenuation is infinite
- The transition between the two is infinitesimally narrow



The Ideal Filter Kernel: the Sinc Function

- The impulse response is the **Inverse Fourier Transform** of the ideal (rectangular) frequency response
- This produces the *sinc* function, defined as

$$\mathrm{sinc}(x) \;=\; \frac{\sin(\pi x)}{\pi x}$$

- The corresponding ideal low-pass filter kernel is

$$h[i] \;=\; 2f_c\ \mathrm{sinc}\big(2f_c\, i\big), \qquad -\infty < i < \infty$$

- Convolving a signal with this kernel gives a **perfect** low-pass filter
- **Problem:** the sinc function extends to $\pm\infty$ without ever reaching zero
  - Not an issue in mathematics — a show-stopper for computers

Step 1: Truncation

- Truncate the infinite sinc to $M + 1$ points, chosen symmetrically around the main lobe
- M is chosen to be an **even** number
- All samples outside this window are set to zero (ignored)

$$h[i] = 0 \quad \text{for } |i| > M/2$$

Step 2: Shifting

- The truncated sequence is shifted right so it runs over indices 0 to M
- This avoids negative array indices, which are awkward in many languages

$$h[i] \;=\; 2f_c\ \mathrm{sinc}\big(2f_c(i - M/2)\big), \qquad i = 0, 1, \ldots, M$$

- The **only** effect of this $M/2$ shift is to delay the output signal by $M/2$ samples

The Cost of Truncation

- The modified kernel is only an **approximation** to the ideal kernel
- Taking the Fourier Transform of the truncated kernel reveals the real frequency response:
  - Excessive ripple in the passband
  - Poor attenuation in the stopband
- Cause: the abrupt discontinuity at the two ends of the truncated sinc (Gibbs phenomenon)

- **Key point:** increasing M (a longer kernel) does *not* fix this — the discontinuity is just as sharp regardless of length

The Windowing Solution

- Multiply the truncated sinc by a smoothly tapering curve called a **window**
- This reduces the abruptness at the truncated ends

$$h_w[i] \;=\; h[i] \cdot w[i]$$

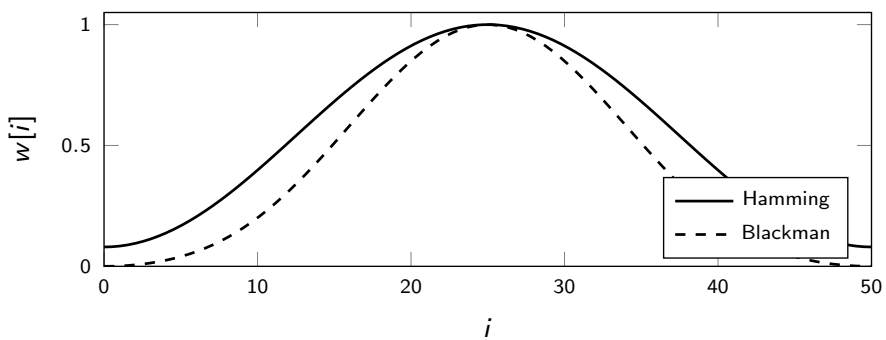
- Result: dramatically improved frequency response
  - Flat passband
  - Stopband attenuation so good it is essentially invisible on a linear graph

Common Window Functions

Only two windows are generally worth using: **Hamming** and **Blackman** (defined for $0 \leq i \leq M$):

$$w_{\text{Hamming}}[i] = 0.54 - 0.46 \cos\left(\frac{2\pi i}{M}\right)$$

$$w_{\text{Blackman}}[i] = 0.42 - 0.5 \cos\left(\frac{2\pi i}{M}\right) + 0.08 \cos\left(\frac{4\pi i}{M}\right)$$



Hamming vs. Blackman: A Trade-off

Window	Roll-off speed	Stopband atten.	Passband ripple
Hamming	$\sim$20% faster than Blackman	-53 dB ($\sim$0.2%)	$\sim$0.2%
Blackman	baseline (slower)	-74 dB ($\sim$0.02%)	$\sim$0.02%

- Hamming rolls off faster, but has worse stopband attenuation
- Blackman rolls off more slowly, but attenuates the stopband far better
- **Rule of thumb:** default to **Blackman** — a slower roll-off is usually easier to live with than poor stopband attenuation

Other Windows (for reference)

Window	Description	Stopband atten.
Bartlett	Triangular, straight-line taper	-25 dB ($\sim$5.6%)
Hanning	Raised cosine: $0.5 - 0.5\cos(2\pi i/M)$	-44 dB ($\sim$0.63%)
Rectangular	No window (plain truncation)	-21 dB ($\sim$8.9%)

- Bartlett and Hanning: roll-off speed similar to Hamming, but worse attenuation
- Rectangular: roll-off $\sim 2.5\times$ faster than Blackman, but by far the worst attenuation
- All three fall short of Hamming and Blackman overall

Characteristics and Performance

- Windowed-sinc filters excel at separating one frequency band from another
- Stable, predictable, and capable of near-ideal frequency-domain performance

- **Trade-off:** poor time-domain behavior
  - Significant ripple and overshoot in the step response

- **Implementation:**
  - Standard convolution $\Rightarrow$ easy to program, but computationally slow
  - FFT-based convolution can dramatically speed up execution for long kernels

Two Parameters to Choose

- Designing a windowed-sinc filter requires choosing **two** parameters:
  - **Cutoff frequency**, f_c — expressed as a fraction of the sampling rate

$$0 < f_c < 0.5$$

  - **Kernel length**, M — must be an **even integer**; sets the roll-off
- M controls the width of the transition band, BW — also expressed as a fraction of the sampling frequency

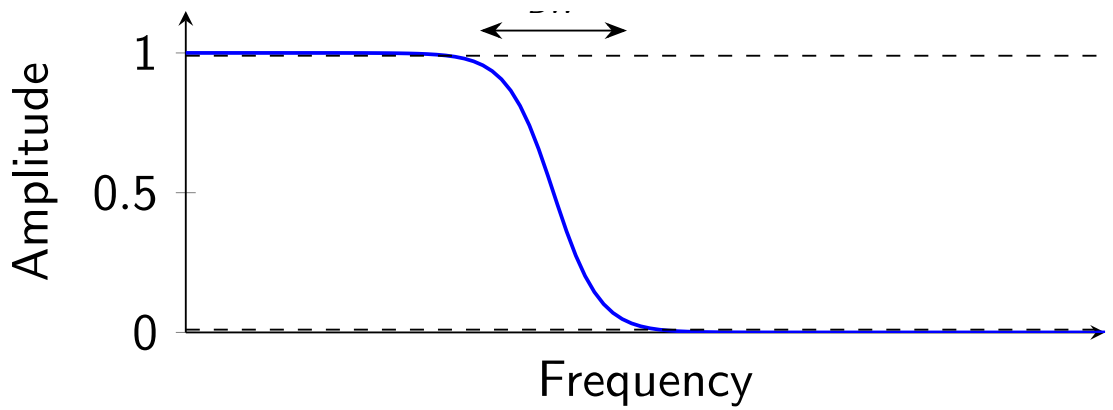
$$0 < BW < 0.5$$

Transition Bandwidth

Approximation (Eq. 16-3)

$$BW \approx \frac{4}{M}$$

- BW is measured from where the curve just leaves 1 to where it almost reaches 0 (conventionally 99% $\rightarrow$ 1%).
- Larger $M \Rightarrow$ narrower transition band $\Rightarrow$ sharper filter.



Example: Effect of M on BW

Three filter kernels, all with the same f_c but different lengths:

M	BW (from Eq. 16-3)	Roll-off
20	0.20	slow
40	0.10	medium
200	0.02	sharp

- Longer kernel $\rightarrow$ narrower transition $\rightarrow$ sharper cutoff.
- The **shape** of the frequency response does not depend on which cutoff frequency f_c was chosen — only on M.

Computation Time vs. Sharpness

- Convolution time is proportional to kernel length M.
- Eq. 16-3 is therefore a trade-off:

$$\underbrace{M}_{\text{compute cost}} \quad \Longleftrightarrow \quad \underbrace{BW}_{\text{filter sharpness}}$$

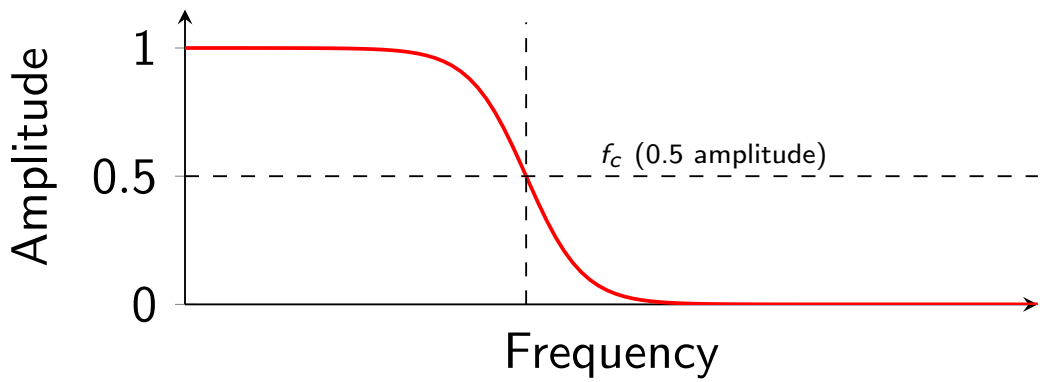
- Example: the Blackman window rolls off about 20% more slowly than the Hamming window.
- This can be compensated by using a kernel about 20% longer — i.e. Blackman is $\sim$20% slower to execute than an equivalent-sharpness Hamming filter.
- Windowed-sinc filters are already computationally expensive — this trade-off matters in practice.

Measuring the Cutoff Frequency

- Cutoff is measured at the **one-half amplitude point** (not the usual $-3\,\text{dB}$ / 0.707 point).
- Reason: the windowed-sinc response is **symmetric** between passband and stopband.

Example — Hamming window

Passband ripple $\approx 0.2\%$ = Stopband ripple $\approx 0.2\%$



Other window types lack this symmetry, so the half-amplitude point has no special advantage for them. This property also makes the windowed-sinc ideal for **spectral inversion**.

The Filter Kernel Equation

Eq. 16-4

$$h[i] = K \cdot \frac{\sin\left(2\pi f_c (i - M/2)\right)}{i - M/2} \cdot \left[0.42 - 0.5\cos\left(\frac{2\pi i}{M}\right) + 0.08\cos\left(\frac{4\pi i}{M}\right)\right]$$

$$i = 0, 1, \ldots, M, \qquad i \neq M/2$$

- Looks intimidating, but decomposes into three familiar pieces (next slide).
- K is a normalizing constant chosen for unity gain at DC.
- In practice: type it into your program once, then forget it — let the computer do the arithmetic.

Breaking Down the Equation

1. **Sinc function:**

$$\frac{\sin(2\pi f_c(i - M/2))}{i - M/2}$$

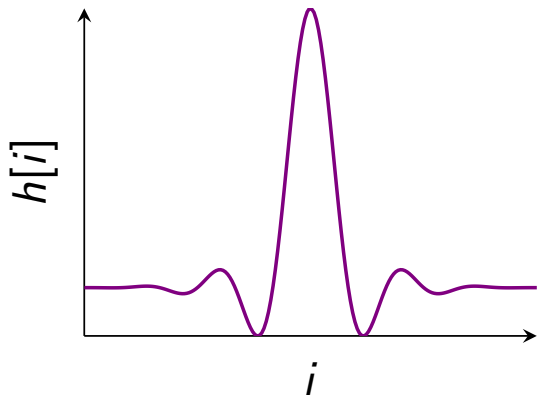
— the ideal low-pass response.

2. $M/2$ **shift:** centers the sinc in the array so the kernel is causal and symmetric.

3. **Blackman window:**

$$0.42 - 0.5\cos\left(\frac{2\pi i}{M}\right) + 0.08\cos\left(\frac{4\pi i}{M}\right)$$

— tapers the ends of the truncated sinc to control ripple.



windowed-sinc kernel

Blackman

Normalizing for Unity Gain

- The constant K must make the kernel sum to exactly 1:

$$\sum_{i=0}^{M} h[i] = 1$$

- This guarantees the filter has **unity gain at DC** (0 Hz is passed unchanged).
- **Practical approach:** ignore K while computing the raw kernel values, then normalize afterward:

$$h_{\text{norm}}[i] = \frac{h[i]}{\sum_j h[j]}$$

Handling the Center Point

- At $i = M/2$, the sinc term becomes $\frac{0}{0}$ — a division by zero.
- This is the removable singularity of the sinc function; its limiting value is used directly:

$$h\left[\frac{M}{2}\right] = K \cdot 2\pi f_c$$

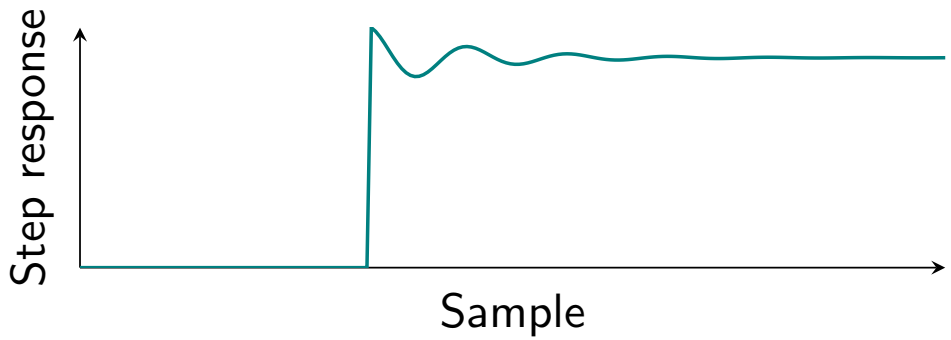
- Any implementation of Eq. 16-4 must special-case this index explicitly.

Array Indexing Example ($M = 100$)

- M must be **even**.
- Kernel occupies array locations 0 through $M \Rightarrow M + 1 = 101$ points total.
- Center of symmetry is at index $M/2 = 50$.
- Symmetry: point 0 = point 100; point 49 = point 51; etc.
- Need a specific length (e.g., for FFT use)? Pad with zeros:
  - $M = 100 \Rightarrow 101$ nonzero samples
  - Add zeros at indices 101–127 $\Rightarrow$ 128-point kernel

Kernel Shape and Step Response

- Samples near the beginning/end of the kernel are tiny — but **not negligible**: collectively they strongly affect filter performance.
- Because kernel values span a huge dynamic range, **floating point** representation is required — integers don't have enough range.



Time-Domain Performance: Terrible

Key limitation

The windowed-sinc step response shows **overshoot** and **ringing**.

- Excellent frequency-domain behavior (sharp, controlled ripple)
- Poor time-domain behavior (ringing, overshoot)
- **Not** suitable for signals where information is encoded in the time domain (e.g., pulse shapes, edges).

Background: EEG Rhythms

- EEG: millivolt-level electrical activity of the brain, measured at scalp electrodes.
- Signal is the combined result of an enormous number of coordinated nerve-cell pulses.
- **Alpha rhythm**: $\approx$7–12 Hz, eyes closed and relaxed — associated with contentment, decreased attention.
- **Beta rhythm**: $\approx$17–20 Hz, eyes open and alert.
- Other frequencies/waveforms appear in children, different sleep depths, and disorders such as epilepsy.

Goal: Separate Alpha from Beta

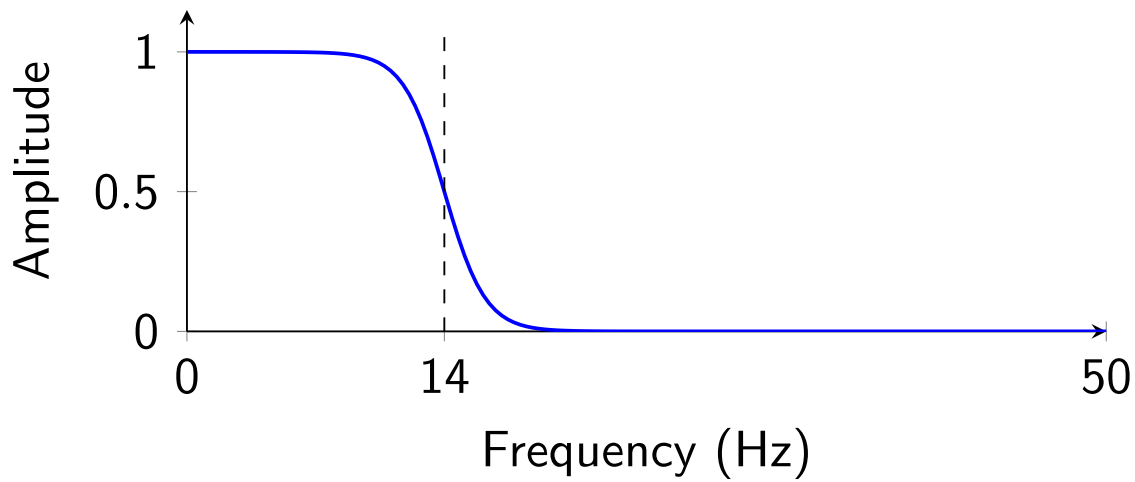
- Signal amplified by analog electronics, then digitized at **100 samples/second**.
- 50 seconds of acquisition $\Rightarrow 50 \times 100 = 5{,}000$ points.
- Design a **low-pass** filter to keep alpha, remove beta.

Filter specification

- Cutoff: $f_c = 14\,\text{Hz} = 0.14$ (fraction of sampling rate)
- Transition band: $BW = 4\,\text{Hz} = 0.04$ (fraction of sampling rate)
- From Eq. 16-3: $M \approx \dfrac{4}{BW} = \dfrac{4}{0.04} = 100 \Rightarrow$ kernel is about **101 points** long
- Window: **Hamming** (chosen arbitrarily here)

Resulting Frequency Response

- Kernel is computed from Eq. 16-4 using the Hamming window, then convolved with the 5,000-point EEG signal.
- Frequency response is obtained by taking the **Fourier Transform** of the filter kernel (Fig. 16-5).



Passes the alpha band (≈7–12 Hz), attenuates the beta band (≈17–20 Hz).

Goal: Isolate a Signaling Tone

- Audio digitized at **10 kHz**.
- Isolate an **80 Hz band** centered on **2 kHz** (e.g., a telephone button tone).
- As fractions of the sampling rate, block everything **below 0.196** and **above 0.204** (i.e. below 1960 Hz and above 2040 Hz).

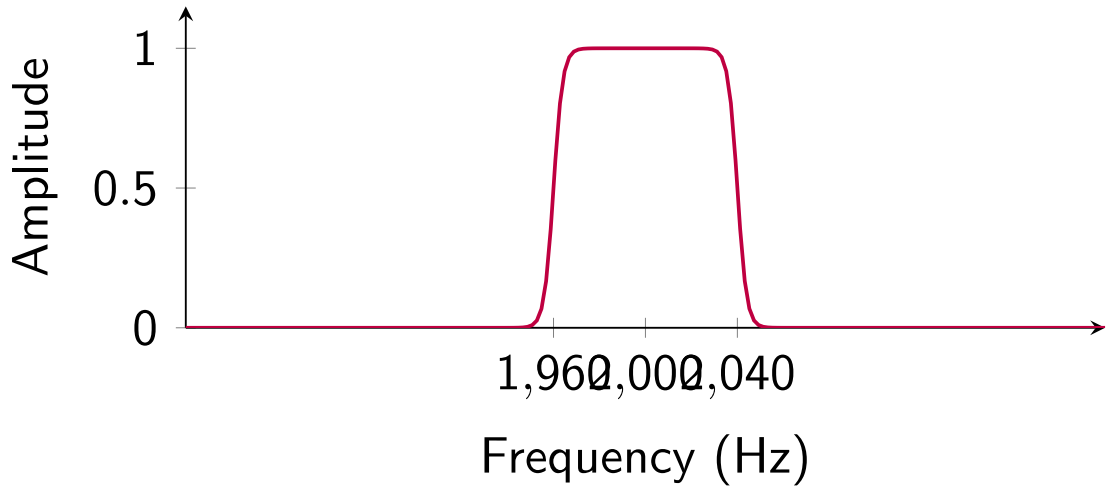
Filter specification

- Transition band: $50\,\text{Hz} = 0.005$ of the sampling rate
- From Eq. 16-3: $M \approx \dfrac{4}{0.005} = 800 \Rightarrow$ kernel is **801 points** long
- Window: **Blackman**

Design Procedure: Build a Band-Pass Filter from Low-Pass Filters

1. Design two low-pass filters: cutoffs at 0.196 and 0.204.
2. Spectrally invert the second $\Rightarrow$ high-pass filter.
3. Add the two kernels $\Rightarrow$ band-**reject** filter (notches out 0.196–0.204).
4. Spectrally invert the result $\Rightarrow$ desired band-**pass** filter.

Resulting Frequency Response



- Narrow 80 Hz passband centered at 2 kHz, steep skirts set by the 801-point Blackman-windowed kernel (Fig. 16-6).
- Long kernel (801 points) is the price paid for the very narrow 0.005 transition band.

Examples: Take-aways

- Same design equations (16-3, 16-4) apply regardless of application — EEG rhythms or telephone tones.
- Narrower transition bandwidth $\Rightarrow$ longer kernel $\Rightarrow$ more computation (100 pts vs. 800 pts here).
- Band-pass (and band-reject) filters are built by combining low-pass filters with **spectral inversion** and **kernel addition** — no new math required.

Extreme Requirement: 1 mV on a 120 V Line

- Goal: isolate a 1 mV signal riding on a 120 V power line.
- Required stopband attenuation: at least -120 dB (one part in one million).
- The Blackman window alone only provides -74 dB (one part in five thousand) — **not enough**.

The problem

A single-stage Blackman windowed-sinc filter falls $\sim$46 dB short of the requirement.

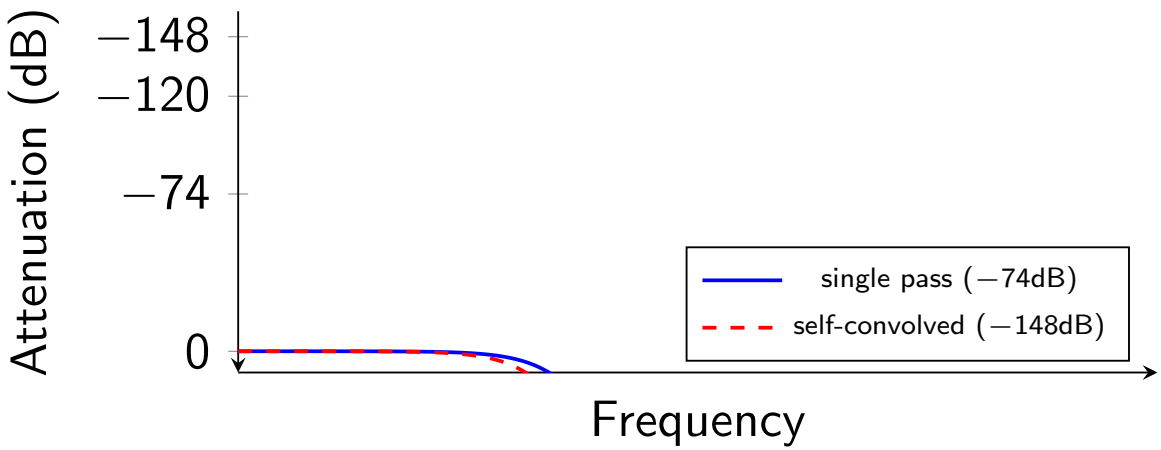
Solution: Cascade the Filter

- Filter the input once $\Rightarrow$ intermediate signal.
- Filter the intermediate signal **again with the same kernel** $\Rightarrow$ stopband attenuation improves to $-\mathbf{148}$ dB (1 part in 30 million!).
- The two passes can be combined into a **single** filter: $h_{\text{combined}} = h * h$ (the kernel convolved with itself).

Price paid: a longer combined kernel and a slower roll-off.

Fig. 16-7a: Self-Convolved Kernel

- A 201-point low-pass filter, formed by convolving a **101-point Blackman** windowed-sinc with itself.



- Convolving *any* filter kernel with itself roughly **doubles** its stopband attenuation in dB.
- Beyond −100 dB, use **double precision** — single precision round-off noise can erratically appear in the stopband at −100 to −120 dB.

Fig. 16-7b: How Far Can You Push It?

A 32,001-point low-pass kernel

- Roll-off of 0.000125 of the sampling rate.
- Passes DC to 1000 Hz with $< 0.02\%$ variation.
- Blocks everything above 1001 Hz with $< 0.02\%$ residue.

- Try building an **analog** electronic filter with that spec — good luck!
- Both Fig. 16-7 examples use only **single precision**.
- Switching to **double precision** extends achievable performance by roughly **a million times**.

The Strongest Limitation: Execution Time

- Standard convolution with a long kernel can be **unacceptably slow**.
- **FFT convolution** (Chapter 18): a high-speed algorithm for computing the same result far faster on long kernels.
- **Recursive filters** (Chapter 19): also give good frequency separation and are a reasonable, much faster alternative to the windowed-sinc filter.

Is Windowed-Sinc Optimal?

- **No** — more sophisticated design techniques can produce better filter kernels.
- But: the windowed-sinc can already deliver **any level of performance you could possibly need**.
- What advanced methods may offer instead:
  - a slightly **shorter** kernel for the same performance
  - therefore, a slightly **faster** execution speed

Word of caution

Before diving into advanced (and heavily mathematical) filter design, weigh the effort against the payoff — it may be modest.